Tech Stack Decisions — AR X-Ray Positioning Tool

This document records the technology choices for the project, the reasoning behind each, and the alternatives that were considered.

1. Pose Detection

Chosen: MediaPipe Pose (Google)

Why:

Runs in real time on CPU — no GPU required, important for a demo that needs to run on a laptop or even in-browser eventually
33 body landmarks out of the box, including shoulders, hips, and face points needed for the positioning checks
Lightweight Python API (`mediapipe` package), minimal setup
Actively maintained by Google, well documented, large community for troubleshooting

Alternatives considered:

Option	Why not chosen
OpenPose	More accurate multi-person detection, but heavier, harder to install (C++/CUDA dependencies), overkill for a single-patient use case
MoveNet (TensorFlow)	Also fast and lightweight, comparable to MediaPipe; viable alternative — mainly not chosen because MediaPipe has more mature Python tooling and better docs for beginners
YOLO-Pose	Good for multi-person scenes and speed, but adds YOLO's detection overhead when only one subject needs tracking; more complex to set up for this scope
Custom-trained model	Would allow domain-specific landmarks (e.g. anatomical positioning points beyond standard body joints), but requires a labeled dataset and training pipeline — not justified for an MVP

2. Computer Vision / Frame Processing

Chosen: OpenCV

Why:

Standard library for reading webcam frames, drawing overlays (skeleton lines, text feedback), and basic image operations
Integrates directly with MediaPipe's output format
Well-known, so it doesn't need separate justification to anyone reviewing the code

Alternatives considered:

Option	Why not chosen
Pillow	Fine for static image editing, but not built for real-time video frame handling
Raw NumPy manipulation	Possible but reinvents what OpenCV already does reliably

3. 3D / AR Visualization Layer

Chosen: Three.js

Why:

Runs in-browser, so the AR overlay is viewable without installing anything beyond a web page
Large ecosystem for 3D rendering, good docs, widely used so it's a transferable skill outside this project
Can render a live 3D skeleton or guide overlay driven by landmark data streamed from Python

Alternatives considered:

Option	Why not chosen
Unity (with AR Foundation)	Much more powerful for true AR/mobile deployment, but heavier tooling, C#, and a build pipeline that's overkill until the 2D logic is proven
A-Frame	Simpler than Three.js for basic AR/VR scenes, but less flexible for custom skeleton rendering logic
Stay in OpenCV only (2D overlay, no 3D)	Valid MVP-only choice — Three.js is deliberately deferred until the 2D version works, per the build plan

4. Communication Between Python (pose logic) and Browser (Three.js)

Chosen: WebSocket

Why:

Pose landmark data needs to stream continuously and with low latency (real-time feedback) — WebSocket keeps a persistent connection open, unlike request/response HTTP
Simple to implement on both ends (`websockets` in Python, native `WebSocket` API in JS)

Alternatives considered:

Option	Why not chosen
REST API polling	Would work but adds request overhead and latency; not suited to 30fps landmark streaming
gRPC	Efficient but adds complexity and tooling not justified at this scale
Direct in-browser pose detection (MediaPipe JS)	Would remove the Python↔JS bridge entirely — worth noting as a future simplification, but the current architecture deliberately keeps Python in the loop since that's where the clinical positioning logic lives and is easier to iterate on

5. Core Language

Chosen: Python (backend/logic) + JavaScript (frontend/visualization)

Why:

Python: best ecosystem for CV/ML libraries (MediaPipe, OpenCV, NumPy), and matches existing skills
JavaScript: required for Three.js/browser-based rendering — no practical alternative for in-browser 3D

Open Questions / Future Decisions

Whether to eventually move pose detection to the browser (MediaPipe JS) to remove the WebSocket bridge and simplify deployment
Whether SID (source-to-image distance) estimation needs depth sensing (e.g. a phone LiDAR) in a later version, since 2D pose landmarks can't measure it
Deployment target: local demo only, or packaged as a web app others can test